

```

"""
sync_atp.py
-----

Incremental daily sync of Sackmann tennis_atp data into Supabase.
Only inserts matches newer than the most recent match already in the DB.
Run via GitHub Actions or manually.
"""

import os
import pandas as pd
from supabase import create_client, Client
from tqdm import tqdm
from datetime import datetime, timedelta

SUPABASE_URL = os.environ["SUPABASE_URL"]
SUPABASE_KEY = os.environ["SUPABASE_KEY"]
DATA_DIR      = "./tennis_atp"
BATCH_SIZE    = 200

supabase: Client = create_client(SUPABASE_URL, SUPABASE_KEY)

SURFACE_MAP = {"Hard": "Hard", "Clay": "Clay", "Grass": "Grass", "Carpet": "Carpet"}
CSI_MAP      = {"Hard": 42.0, "Clay": 28.0, "Grass": 55.0, "Carpet": 50.0}
TIER_MAP     = {"G": "Grand Slam", "M": "Masters 1000", "A": "ATP 500",
                "D": "ATP 250", "250": "ATP 250", "500": "ATP 500", "C": "Challenger", "F"}

def dedup(rows, key):
    seen, out = set(), []
    for r in rows:
        if r[key] not in seen:
            seen.add(r[key])
            out.append(r)
    return out

def batch_upsert(table, rows, on_conflict):
    rows = dedup(rows, on_conflict)
    for i in range(0, len(rows), BATCH_SIZE):
        supabase.table(table).upsert(rows[i:i+BATCH_SIZE], on_conflict=on_conflict)

def si(v):
    try: return int(v) if pd.notna(v) else None
    except: return None

def sf(num, den):

```

```

        try: return round(float(num)/float(den)*100,2) if pd.notna(num) and pd.notna(
        except: return None

# — Step 1: Find the most recent match date already in DB —————
print("Checking last sync date...")
result = supabase.table("matches").select("match_date").order("match_date", desc=
if result.data and result.data[0]["match_date"]:
    last_date = datetime.strptime(result.data[0]["match_date"], "%Y-%m-%d")
    # Go back 7 days to catch any late-reported matches
    cutoff = (last_date - timedelta(days=7)).strftime("%Y%m%d")
    cutoff_year = last_date.year
    print(f"  Last match in DB: {last_date.date()} – syncing from {cutoff}")
else:
    cutoff = "20150101"
    cutoff_year = 2015
    print("  No matches found – doing full load from 2015")

# — Step 2: Load only current + previous year CSVs —————
current_year = datetime.now().year
frames = []
for year in range(max(cutoff_year, current_year - 1), current_year + 1):
    path = os.path.join(DATA_DIR, f"atp_matches_{year}.csv")
    if os.path.exists(path):
        df = pd.read_csv(path, low_memory=False)
        df["year"] = year
        frames.append(df)

if not frames:
    print("No new CSV data found. Exiting.")
    exit(0)

df = pd.concat(frames, ignore_index=True)

# Filter to only rows newer than cutoff
df = df[df["tourney_date"].astype(str) >= cutoff]
print(f"  {len(df):,} new match rows to process")

if len(df) == 0:
    print("Database is already up to date.")
    exit(0)

# — Step 3: Upsert players —————
print("\nUpserting players...")
winners = df[["winner_name", "winner_hand", "winner_ht", "winner_ioc"]].rename(
    columns={"winner_name": "name", "winner_hand": "hand", "winner_ht": "height_cm", "w
losers = df[["loser_name", "loser_hand", "loser_ht", "loser_ioc"]].rename(
    columns={"loser_name": "name", "loser_hand": "hand", "loser_ht": "height_cm", "lose

```

```

players = pd.concat([winners, losers]).drop_duplicates(subset=["name"]).dropna(su

player_rows = []
for _, p in players.iterrows():
    player_rows.append({
        "name": str(p["name"]),
        "hand": p["hand"] if p["hand"] in ("R", "L", "U") else "U",
        "height_cm": int(p["height_cm"]) if pd.notna(p["height_cm"]) else None,
        "nationality": str(p["nationality"]) if pd.notna(p["nationality"]) else N
    })
batch_upsert("players", player_rows, "name")
result = supabase.table("players").select("player_id, name").execute()
player_map = {r["name"]: r["player_id"] for r in result.data}
print(f" {len(player_map):,} total players in DB")

# — Step 4: Upsert tournaments —————
print("\nUpserting tournaments...")
tours = df[["tourney_id", "tourney_name", "surface", "tourney_level", "draw_size"]].d
tour_rows = []
for _, t in tours.iterrows():
    surface = SURFACE_MAP.get(str(t["surface"]), "Hard")
    tour_rows.append({
        "name": str(t["tourney_name"]),
        "surface": surface,
        "court_speed_index": CSI_MAP.get(surface),
        "draw_size": int(t["draw_size"]) if pd.notna(t["draw_size"]) else
        "tier": TIER_MAP.get(str(t["tourney_level"]), "ATP 250"),
    })
batch_upsert("tournaments", tour_rows, "name")
result = supabase.table("tournaments").select("tournament_id, name").execute
name_to_id = {r["name"]: r["tournament_id"] for r in result.data}
tourney_map = {t["tourney_id"]: name_to_id.get(str(t["tourney_name"])) for _, t i
print(f" {len(tourney_map):,} total tournaments in DB")

# — Step 5: Insert new matches + stats —————
print("\nInserting new matches and stats...")
match_rows = []
valid_idx = []
for idx, row in df.iterrows():
    w_id = player_map.get(str(row.get("winner_name")))
    l_id = player_map.get(str(row.get("loser_name")))
    t_id = tourney_map.get(row.get("tourney_id"))
    if not w_id or not l_id or not t_id: continue
    raw = str(row.get("tourney_date", ""))
    match_rows.append({
        "tournament_id": t_id, "round": str(row.get("round", "")),
        "match_date": f"{raw[:4]}-{raw[4:6]}-{raw[6:]}" if len(raw)==8 else No

```

```

        "winner_id":      w_id, "loser_id": l_id,
        "score":         str(row.get("score")) if pd.notna(row.get("score")) else
        "retirement":   str(row.get("score","")).endswith("RET"),
        "best_of":       int(row["best_of"]) if pd.notna(row.get("best_of")) else
    })
    valid_idx.append(idx)

df_valid = df.loc[valid_idx].reset_index(drop=True)
stat_rows = []

for i in tqdm(range(0, len(match_rows), BATCH_SIZE), desc="Matches"):
    result = supabase.table("matches").insert(match_rows[i:i+BATCH_SIZE]).execute()
    inserted = result.data
    batch = df_valid.iloc[i:i+BATCH_SIZE].reset_index(drop=True)
    for k, match in enumerate(inserted):
        if k >= len(batch): break
        row = batch.iloc[k]
        match_id = match["match_id"]
        w_id = player_map.get(str(row.get("winner_name")))
        l_id = player_map.get(str(row.get("loser_name")))
        svpt_w = row.get("w_svpt"); in_w = row.get("w_1stIn")
        svpt_l = row.get("l_svpt"); in_l = row.get("l_1stIn")
        stat_rows += [
            {"match_id":match_id,"player_id":w_id,
             "aces":si(row.get("w_ace")), "double_faults":si(row.get("w_df")),
             "first_serve_pct":sf(in_w,svpt_w),
             "first_serve_won_pct":sf(row.get("w_1stWon"),in_w),
             "second_serve_won_pct":sf(row.get("w_2ndWon"),(si(svpt_w) or 0)-(si(
             "bp_faced":si(row.get("w_bpFaced")), "bp_saved":si(row.get("w_bpSaved
             "service_games_played":si(row.get("w_SvGms")))},
            {"match_id":match_id,"player_id":l_id,
             "aces":si(row.get("l_ace")), "double_faults":si(row.get("l_df")),
             "first_serve_pct":sf(in_l,svpt_l),
             "first_serve_won_pct":sf(row.get("l_1stWon"),in_l),
             "second_serve_won_pct":sf(row.get("l_2ndWon"),(si(svpt_l) or 0)-(si(
             "bp_faced":si(row.get("l_bpFaced")), "bp_saved":si(row.get("l_bpSaved
             "service_games_played":si(row.get("l_SvGms")))},
        ]

for i in tqdm(range(0, len(stat_rows), BATCH_SIZE), desc="Stats  "):
    supabase.table("match_stats").insert(stat_rows[i:i+BATCH_SIZE]).execute()

print(f"""
 Sync complete!
    New matches:  {len(match_rows):,}
    New stats:    {len(stat_rows):,}
""")

```

